



DoubleVariable

🕒 Created time	June 20, 2026 1:06 PM
☰ Category	CommonVariables
👤 Last edited by	👤 R.A R.A
🕒 Last updated time	July 4, 2026 4:53 AM

```
//Doubles can have a Decimal point - 1.23, 43

double averageAge;

//averageAge = (43.0 + 22 + 62) / 3;

averageAge = 2.3 / 3;

Console.WriteLine(averageAge);
```

ה-Double מעט שונה מ-Int מכיוון ש-Double יכול להכיל נקודה עשרונית.

כמו 1.23 ו-43, אנחנו שואלים למה 43, הרי אין לו נקודה עשרונית.

התשובה היא: מבחינה טכנית יכולה להיות לו נקודה עשרונית - 43.0.

אבל כך או כך זה בסדר, לא חייבת להיות לו נקודה עשרונית, הוא לא מחויב לנקודה עשרונית, אבל אנחנו יכולים לשלב נקודה עשרונית ב-Double שלנו.

בסביבת הפיתוח של #C, משתנה מסוג Int נועד לאחסן אך ורק מספרים שלמים.

לעומת זאת, משתנה מסוג Double נועד לאחסן מספרים עשרוניים, אבל מציע גמישות רחבה יותר.

הקומפילר של #C יודע להתמודד עם מספרים שלמים גם כשהם נכנסים לתוך משתנה שמיועד לשברים.

כשמכניסים את המספר 43 לתוך משתנה מסוג Double, המערכת מבצעת המרה אוטומטית ושקופה מאחורי הקלעים ומתייחסת למספר כאל 43.0.

המשמעות בפועל היא שלא חייבים לכתוב את הנקודה העשרונית בקוד אם הערך ההתחלתי הוא שלם לחלוטין. המשתנה יקבל את המספר בשמחה, ולא יחייב למבנה של שבר, אך יחד עם זאת ישמור על היכולת לאחסן שברים (כמו 1.23) באותו משתנה בדיוק אם הערך שלו ישתנה בהמשך ריצת התוכנית.

זה מאפשר כתיבת קוד נקייה יותר ומניעת שגיאות כשעובדים עם נתונים דינמיים.

~~~~~

```
averageAge = (43 + 21 + 62) / 3;  
averageAge = (43.0 + 21 + 62) / 3;
```

התוצאה של הקוד הזה תהיה מסוג Int (מספר שלם): ממוצע הגילאים  $(62 + 21 + 43) / 3$ .  
אבל אם נכתוב את אותו הקוד כך: ממוצע הגילאים  $(62 + 21 + 43.0) / 3$ .  
עכשיו זה מסוג Double (מספר עשרוני), והתוצאה של הקוד הזה תהיה עשרונית ותהיה שווה ל-42.333333333333336.  
הקומפיילר מניח שמדובר ב-Int אם הם מספרים שלמים.

אבל אם נוסיף 0. (למשל 43.0), שזה עדיין בדיוק אותו מספר מבחינת הערך שלו, הקומפיילר יראה את זה כסוג Double. בעקבות זאת, הקומפיילר יראה גם את שאר המספרים ואת התוצאה שלהם כ-Double.  
לכן, אנחנו צריכים לתת לקומפיילר "רמז" אם אנחנו רוצים שהתוצאה תהיה Double.

בשפת C#, כאשר מבצעים פעולות מתמטיות, הקומפיילר עובד לפי סוגי הנתונים (Types) שמספקים לו.  
אם כל המספרים במשוואה הם מסוג Int (מספרים שלמים), הקומפיילר יבצע מה שנקרא "חילוק שלמים" (Integer Division).  
המשמעות היא שהוא יחשב את התוצאה, אבל יזרוק לפח את כל השארית שאחרי הנקודה העשרונית ויחזיר רק מספר שלם, מה שמוביל לאובדן מידע ולתוצאה לא מדויקת.  
כדי לתקן את זה, אנחנו צריכים לעבור למתמטיקה של מספרים עשרוניים.  
**אחת השיטות הכי נפוצות ויעילות:** מספיק להפוך רק מספר אחד במשוואה ל-Double כדי לשנות את כל אופן החישוב.

ברגע שמוסיפים את ה-0. למספר 43, אנחנו בעצם נותנים לקומפיילר את הרמז שהוא צריך.  
הקומפיילר רואה את 43.0 ומזהה אותו מיד כ-Double.

**בשפת C# יש חוק שנקרא "קידום סוגים" (Type Promotion):** ברגע שפעולה מתמטית מערבת משתנה ברמת דיוק גבוהה (כמו Double) ומשתנים ברמת דיוק נמוכה יותר (כמו Int), הקומפיילר ממיר באופן אוטומטי מאחורי הקלעים את כל שאר ה-Ints שבמשוואה ל-Double כדי שיוכלו לדבר באותה שפה.  
כך, כל החישוב מתבצע ברמה העשרונית, והתוצאה הסופית נשמרת במלואה, כולל כל הספרות שאחרי הנקודה.

~~~~~

שימו לב שמספר שלם (Integer) יכול להיכנס לתוך Double ללא בעיה, מכיוון שהוא עובר מסוג פחות ספציפי ליותר ספציפי.

לכן 43 יכול להתאים ל-Integer כמספר שלם, ו-43 יכול להתאים ל-Double בלי לשנות שום דבר, כי 43 זה 43. אבל אם היינו עושים את הכיוון ההפוך ומנסים להכניס Double לתוך Integer, כמו המספר 1.23, היינו צריכים לחתוך ממנו את ה-23. וזה לא היה עובד כי נאבד מידע.
לכן אי אפשר להכניס Double ל-Integer, אפילו אם ה-Double הזה הוא סתם 43, כי המערכת לא יודעת שזה תמיד יהיה המצב.

המפתח כאן הוא של-Doubles יש נקודות עשרוניות, או שהם יכולים לכלול נקודות עשרוניות.

אם נחבר 3 מספרים כמו $62 + 21 + 43$ ונחלק ב-3, בגלל שכולם מונחים להיות Integers על ידי הקומפיילר, התוצאה תהיה ערך Integer שנכנס לתוך Double, אבל זה מאבד את הנקודה העשרונית שלנו באותה חלוקה בגלל השימוש ב-Integers.

אבל אם נציין שאחד מהם הוא Double, השאר ילכו בעקבותיו וזה יהפוך לפעולת Double, זה ישמור על הערכים העשרוניים שלנו.

שני עקרונות קריטיים בניהול זיכרון וטיפוסי נתונים (Data Types) בשפת C#, שחשוב מאוד להבין כדי לכתוב קוד נקי ובטוח.

המרת נתונים בטוחה מול המרה מסוכנת (Casting)

כאשר מעבירים נתון מסוג int אל double, התהליך קורה אוטומטית (Implicit Casting) כי double הוא "מיכל" גדול ומדויק יותר.

לכן המערכת מבינה שאין שום סכנה לאיבוד מידע.

לעומת זאת, ניסיון להכניס double לתוך int נחסם באופן גורף על ידי הקומפיילר.

המערכת לא לוקחת סיכונים: מכיוון ש-int לא יודע להחזיק שברים, כל ערך עשרוני יחתך ויימחק.

הקומפיילר יחסום את הפעולה הזו מראש אפילו אם המספר העשרוני הוא בדיוק 43.0, כדי להגן מפני אובדן מידע שעלול לרסק מערכות אמיתיות.

פעולות מתמטיות ואיבוד שארית (Integer Division)

כאשר כותבים חישוב מתמטי שכולל רק מספרים שלמים, הקומפיילר של C# מתייחס לכל החישוב כפעולה של int. המשמעות היא שאם תוצאת החילוק כוללת שבר (כמו 42.333), השבר הזה ייזרק לפח עוד לפני שהתוצאה נשמרת. הפתרון המקצועי הוא לאלץ את הקומפיילר להתייחס לחישוב כפעולה עשרונית.

ברגע שמגדירים לפחות אחד מהמספרים בפעולה כ-double, כל שאר המספרים בחישוב מיישרים קו, החישוב מתבצע ברמת הדיוק הגבוהה ביותר, והשארית נשמרת במלואה.

~~~~~

העניין המרכזי עם משתנים מסוג Double הוא שהם מכילים נקודה עשרונית, ולכן הם הסוג המספרי הנפוץ ביותר כשמדובר במתמטיקה.

עם משתני Double אנחנו יכולים לחבר, לחסר, להכפיל, לחלק, להוציא שורש ריבועי, ובגדול לבצע כל פעולה שנרצה. לכן, בכל מה שקשור לחישובים מתמטיים, אנחנו נשתמש ב-Doubles.

השוואה של ה-Double ל-Int.

משתנה מסוג Int מסוגל לשמור רק מספרים שלמים, מה שמגביל אותנו ברגע שאנחנו מבצעים פעולות מתמטיות אמיתיות.

אם נחלק שני מספרים שלמים, למשל 5 לחלק ל-2, התוצאה היא 2.5.

משתנה מסוג Int יזרוק את ה-5 לפח וישאיר אותנו רק עם 2, וזה מצב שאנחנו לא יכולים להרשות לעצמנו בחישובים מדויקים.

העניין הקריטי הוא היכולת הזו לשמור על השבר.

בעולם הפיתוח המודרני של 2026 בסביבת דוט נט, כשאנחנו כותבים קוד למערכות מורכבות, חישובים מתמטיים דורשים גמישות ודיוק.

בגלל ש-Double תומך בנקודה עשרונית, הוא הפך להיות טיפוס הנתונים הסטנדרטי (ברירת המחדל) לרוב הפעולות המתמטיות כמו חילוק, הוצאת שורש או טריגונומטריה, שבהן כמעט תמיד נקבל תוצאה לא שלמה. הוא פשוט מאפשר לתוכנה שלנו להתנהג כמו מחשבון אמיתי בלי לאבד מידע בדרך.

~~~~~

```
averageAge = 2.3 / 3;
```

מתמטיקה ב-.NET: כתבו **Math**.

זה יפתח את הספרייה.

כל הפעולות שם עובדות עם Doubles, מכיוון שזהו הסוג הנפוץ ביותר לאחסון מספרים עם נקודה עשרונית.

זה גם הטיפוס הזול ביותר, והסיבה לכך קשורה למקום שבו הוא מעגל את הערכים.

הם לא סופר מדויקים, יש קצת עיגול ב-Double שלנו, ויש קצת עיגול בכל טיפוס, ככה זה עובד.

אבל זה קצת יותר יעיל מטיפוסים אחרים כי יש לו קצת פחות ספרות משמעותיות.

אנחנו יכולים להשתמש בזה כמעט לכל פעולה כשמדובר במספרים עשרוניים.

היוצא מן הכלל יהיה כסף - אנחנו לא משתמשים ב-Doubles כדי לאחסן כסף, כי אנחנו רוצים לקבל דיוק גבוה יותר כשמדובר בכסף.

ספריית המתמטיקה של שפת #C

כאשר מקלידים בסביבת הפיתוח פקודה **Math** (עם נקודה), המערכת פותחת גישה לספרייה שלמה של פעולות מתמטיות מובנות מראש, כמו חישוב חזקות, שורשים ועיגול מספרים.

רוב הפעולות האלו דורשות או מחזירות משתנים מסוג **double**.

מיקרוסופט בנתה את זה ככה כי בעולם התוכנה, זהו סוג הנתונים הסטנדרטי והנפוץ ביותר לעבודה עם שברים וחישובים מורכבים.

למה הטיפוס הזה נחשב לזול ויעיל

תיאור ה-Double כמשתנה זול, לא מדובר על כסף, אלא על המשאבים של המעבד (CPU).

למעבדים מודרניים קל ומהיר מאוד לבצע פעולות עם משתני **double** (המוכרים בתעשייה גם כמספרי נקודה צפה). הטיפוס הזה צורך פחות כוח עיבוד וזמן ריצה בהשוואה לטיפוסים אחרים שמיועדים למספרים עשרוניים, מה שהופך את הקוד למהיר ויעיל יותר.

מחיר המהירות הוא חוסר דיוק מוחלט

היעילות הזו מגיעה עם פשרה קטנה.

משתנה מסוג `double` שומר בערך 15 עד 17 ספרות אחרי הנקודה.

בקצוות הארוכים האלה, המערכת מתחילה לעגל את המספר כדי לחסוך בזיכרון, מה שיכול ליצור סטיות מתמטיות קטנטנות בחישוב (למשל, תוצאה של `0.30000000000000004` במקום `0.3` עגול).

ברוב המוחלט של המערכות, סטייה מיקרוסקופית כזו לא מעניינת אף אחד, ולכן נעדיף את המהירות של ה-`Double` על פני דיוק אבסולוטי.

מתי אסור להשתמש בזה

מפתחים חייבים להכיר את החריג היחיד והקריטי ביותר: כסף.

לעולם לא נשתמש ב-`double` כשכותבים קוד של חנות אינטרנטית, מערכת שכר או אפליקציה פיננסית.

אותן סטיות עיגול קטנטנות יכולות להצטבר לחוסרים של שקלים שלמים כשיש אלפי פעולות וחישובי מס.

עבור נתונים פיננסיים אנחנו נשתמש תמיד בטיפוס אחר שנקרא `decimal`, שהוא איטי יותר ויקר יותר למעבד, אבל מספק רמת דיוק עליונה שלא תאבד למערכת אפילו אגורה אחת.